

oldver\trainex3.py

```
1
2 import sys, time, json, subprocess, copy
3 import numpy as np
4 import pandas as pd
5 import torch
6 import torch.nn as nn
7 import matplotlib
8 matplotlib.use("Agg")
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11 from sklearn.metrics import (classification_report, confusion_matrix, roc_curve,
12                             auc, precision_recall_fscore_support, f1_score)
13 from sklearn.preprocessing import label_binarize
14
15 from config3 import (
16     seed_everything, SEED, DEVICE, OUTPUT_DIR, FIGURES_DIR,
17     NUM_CLASSES, CLASS_NAMES, BATCH_SIZE, EPOCHS, WARMUP_EPOCHS, LEARNING_RATE,
18     WEIGHT_DECAY, PATIENCE, LABEL_SMOOTHING, GRAD_ACCUM_STEPS, IMAGE_SIZE,
19     USE_AUX, AUX_WEIGHT, USE_MIXUP, MIXUP_ALPHA, COMPARE_SE,
20     MODEL_SAVE_PATH, MODEL_SAVE_PATH_ROOT, HISTORY_SAVE_PATH, UTIL_SAVE_PATH,
21 )
22 from dataset3 import get_data loaders, val_transform, VirusDataset
23 from model3 import build_model
24 from torch.utils.data import DataLoader
25
26
27 def fmt_time(s): m, s = divmod(int(s), 60); return f"{m}m {s:02d}s" if m else f"{s}s"
28
29
30 # — mixup helper —————
31 def mixup_batch(x, y, alpha):
32     lam = np.random.beta(alpha, alpha) if alpha > 0 else 1.0
33     idx = torch.randperm(x.size(0), device=x.device)
34     return lam * x + (1 - lam) * x[idx], y, y[idx], lam
35
36
37 # — one training epoch (handles AMP, aux heads, mixup, grad-accum) —————
38 def train_one_epoch(model, loader, criterion, optimizer, scaler, use_amp):
39     model.train()
40     run_loss, correct, total = 0.0, 0, 0
41     optimizer.zero_grad(set_to_none=True)
42     for step, (images, labels) in enumerate(loader):
43         images = images.to(DEVICE, non_blocking=True)
44         labels = labels.to(DEVICE, non_blocking=True)
45         if USE_MIXUP:
46             images, ya, yb, lam = mixup_batch(images, labels, MIXUP_ALPHA)
47
48         with torch.amp.autocast(DEVICE.type, enabled=use_amp):
49             out = model(images)
50             heads = out if isinstance(out, tuple) else (out,)
51             weights = [1.0] + [AUX_WEIGHT] * (len(heads) - 1)
52             if USE_MIXUP:
53                 loss = sum(w * (lam * criterion(h, ya) + (1 - lam) * criterion(h, yb))
54                           for w, h in zip(weights, heads))
55             else:
56                 loss = sum(w * criterion(h, labels) for w, h in zip(weights, heads))
57             loss = loss / GRAD_ACCUM_STEPS
58
59             scaler.scale(loss).backward()
60             if (step + 1) % GRAD_ACCUM_STEPS == 0:
61                 scaler.step(optimizer); scaler.update()
62                 optimizer.zero_grad(set_to_none=True)
63
64             main = heads[0]
65             run_loss += loss.item() * images.size(0) * GRAD_ACCUM_STEPS
66             pred = main.argmax(1)
67             total += labels.size(0)
68             correct += pred.eq(labels).sum().item() # approx acc when mixup is on
69     return run_loss / total, 100.0 * correct / total
70
```

```

71
72 @torch.no_grad()
73 def validate(model, loader, criterion, use_amp):
74     model.eval()
75     run_loss, correct, total = 0.0, 0, 0
76     L, P, PR = [], [], []
77     for images, labels in loader:
78         images = images.to(DEVICE, non_blocking=True)
79         labels = labels.to(DEVICE, non_blocking=True)
80         with torch.amp.autocast(DEVICE.type, enabled=use_amp):
81             logits = model(images)
82             loss = criterion(logits, labels)
83             probs = torch.softmax(logits.float(), 1)
84             pred = logits.argmax(1)
85             run_loss += loss.item() * images.size(0)
86             total += labels.size(0); correct += pred.eq(labels).sum().item()
87             L.append(labels.cpu().numpy()); P.append(pred.cpu().numpy()); PR.append(probs.cpu().numpy())
88     return (run_loss / total, 100.0 * correct / total,
89           np.concatenate(L), np.concatenate(P), np.vstack(PR))
90
91
92 # — training loop for ONE variant (use_se True/False) —————
93 def train_variant(use_se, train_loader, val_loader, train_df, val_df, tag):
94     seed_everything(SEED)
95     model = build_model(use_se=use_se, use_aux=USE_AUX).to(DEVICE)
96     total_p, train_p = model.count_parameters()
97     criterion = nn.CrossEntropyLoss(label_smoothing=LABEL_SMOOTHING)
98     optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE,
99                                   weight_decay=WEIGHT_DECAY)
100     warmup = torch.optim.lr_scheduler.LinearLR(optimizer, start_factor=0.1,
101                                                total_iters=WARMUP_EPOCHS)
102     cosine = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
103                                                         T_max=max(1, EPOCHS - WARMUP_EPOCHS))
104     scheduler = torch.optim.lr_scheduler.SequentialLR(
105         optimizer, [warmup, cosine], milestones=[WARMUP_EPOCHS])
106     use_amp = (DEVICE.type == "cuda")
107     scaler = torch.amp.GradScaler(DEVICE.type, enabled=use_amp)
108
109     gpu = torch.cuda.get_device_name(0) if use_amp else "CPU"
110     print("\n" + "=" * 92)
111     print(f"  TRAINING [{tag}] | SE={use_se} | {len(train_df)} train / {len(val_df)} val")
112     print("=" * 92)
113     print(f"  Device {gpu} | Params {total_p:,} | LR {LEARNING_RATE} | Batch {BATCH_SIZE}"
114           f" | Epochs {EPOCHS} | Mixup {USE_MIXUP}")
115     print("=" * 92)
116
117     best_acc, patience_ctr, history = 0.0, 0, []
118     best_state = copy.deepcopy(model.state_dict())
119     t0 = time.time()
120     for ep in range(1, EPOCHS + 1):
121         te = time.time()
122         tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer, scaler, use_amp)
123         va_loss, va_acc, *_ = validate(model, val_loader, criterion, use_amp)
124         scheduler.step()
125         lr = optimizer.param_groups[0]["lr"]; et = time.time() - te
126         marker = ""
127         if va_acc > best_acc:
128             best_acc, patience_ctr = va_acc, 0
129             best_state = copy.deepcopy(model.state_dict())
130             best_epoch = ep; marker = " * Best"
131         else:
132             patience_ctr += 1
133         history.append({"epoch": ep, "train_loss": tr_loss, "train_acc": tr_acc,
134                       "val_loss": va_loss, "val_acc": va_acc, "lr": lr, "time_s": et})
135         print(f"  Ep {ep>3}/{EPOCHS} | Train L {tr_loss:.4f} A {tr_acc:6.2f}% | "
136               f"Val L {va_loss:.4f} A {va_acc:6.2f}% | LR {lr:.1e} | "
137               f"{fmt_time(et)} ETA {fmt_time(et*(EPOCHS-ep))}{marker}")
138         if patience_ctr >= PATIENCE:
139             print(f"  Early stopping at epoch {ep} (no val gain for {PATIENCE}).")
140             break
141     train_time = time.time() - t0
142     model.load_state_dict(best_state)

```

```

143     print(f" [{tag}] done in {fmt_time(train_time)} | best Val {best_acc:.2f}% @ ep {best_epoch}")
144     return model, pd.DataFrame(history), {
145         "tag": tag, "use_se": use_se, "best_val_acc": best_acc,
146         "params": total_p, "best_epoch": best_epoch, "train_time_s": train_time,
147     }
148
149
150 # — speed + memory benchmark —————
151 @torch.no_grad()
152 def benchmark(model):
153     model.eval()
154     x1 = torch.randn(1, 3, IMAGE_SIZE, IMAGE_SIZE, device=DEVICE)
155     for _ in range(10): _ = model(x1) # warmup
156     if DEVICE.type == "cuda":
157         torch.cuda.synchronize(); torch.cuda.reset_peak_memory_stats()
158     t = time.time()
159     for _ in range(50): _ = model(x1)
160     if DEVICE.type == "cuda": torch.cuda.synchronize()
161     lat_ms = (time.time() - t) / 50 * 1000
162     peak_mb = (torch.cuda.max_memory_allocated() / 1e6) if DEVICE.type == "cuda" else 0.0
163     total_p, _ = model.count_parameters()
164     size_mb = total_p * 4 / 1e6
165     return {"latency_ms": lat_ms, "fps": 1000.0 / lat_ms,
166           "peak_vram_mb": peak_mb, "model_size_mb": size_mb, "params": total_p}
167
168
169 # — plots (learning curve / confusion / ROC / per-class) —————
170 def plot_learning_curves(hist, tag):
171     fig, (a, b) = plt.subplots(1, 2, figsize=(14, 5))
172     a.plot(hist.epoch, hist.train_loss, "o-", label="train"); a.plot(hist.epoch, hist.val_loss, "o-", label="val")
173     a.set_title(f"Loss [{tag}]"); a.set_xlabel("epoch"); a.legend(); a.grid(alpha=.3)
174     b.plot(hist.epoch, hist.train_acc, "o-", label="train"); b.plot(hist.epoch, hist.val_acc, "o-", label="val", color="g")
175     b.set_title(f"Accuracy [{tag}]"); b.set_xlabel("epoch"); b.legend(); b.grid(alpha=.3)
176     plt.tight_layout(); p = FIGURES_DIR / f"learning_curves_{tag}.png"
177     plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ok {p.name}")
178
179 def plot_confusion(y, yp, tag, split):
180     cm = confusion_matrix(y, yp)
181     for norm, cmap, fmt, name in [(False, "Blues", "d", "cm"), (True, "Oranges", ".1f", "cmnorm")]:
182         M = cm.astype(float) / cm.sum(1, keepdims=True) * 100 if norm else cm
183         plt.figure(figsize=(7, 6))
184         sns.heatmap(M, annot=True, fmt=fmt, cmap=cmap, xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
185         plt.xlabel("Predicted"); plt.ylabel("Truth"); plt.title(f"{name} {split} [{tag}]")
186         plt.tight_layout(); p = FIGURES_DIR / f"{name}_{split}_{tag}.png"
187         plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ok {p.name}")
188
189 def plot_roc(y, prob, tag):
190     yb = label_binarize(y, classes=list(range(NUM_CLASSES)))
191     plt.figure(figsize=(7, 6))
192     for i, c in enumerate(CLASS_NAMES):
193         fpr, tpr, _ = roc_curve(yb[:, i], prob[:, i]); plt.plot(fpr, tpr, lw=2, label=f"{c} (AUC {auc(fpr,tpr):.3f})")
194     plt.plot([0, 1], [0, 1], "k--", alpha=.5); plt.xlabel("FPR"); plt.ylabel("TPR")
195     plt.title(f"ROC OvR [{tag}]"); plt.legend(loc="lower right"); plt.grid(alpha=.3)
196     plt.tight_layout(); p = FIGURES_DIR / f"roc_{tag}.png"
197     plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ok {p.name}")
198
199 def plot_per_class(y, yp, tag):
200     pr, rc, f1, _ = precision_recall_fscore_support(y, yp, labels=list(range(NUM_CLASSES)), zero_division=0)
201     x = np.arange(NUM_CLASSES); w = .25
202     plt.figure(figsize=(10, 5))
203     plt.bar(x - w, pr, w, label="Precision"); plt.bar(x, rc, w, label="Recall"); plt.bar(x + w, f1, w, label="F1")
204     plt.xticks(x, CLASS_NAMES); plt.ylim(0, 1.15); plt.title(f"Per-class [{tag}]"); plt.legend(); plt.grid(axis="y", alpha=.3)
205     plt.tight_layout(); p = FIGURES_DIR / f"per_class_{tag}.png"
206     plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ok {p.name}")
207
208
209 def full_evaluation(model, train_eval_loader, val_loader, hist, tag):
210     use_amp = (DEVICE.type == "cuda"); crit = nn.CrossEntropyLoss()
211     print("\n" + "=" * 92); print(f" EVALUATION [{tag}]"); print("=" * 92)
212     plot_learning_curves(hist, tag)
213     vl, va, yl, yp, pr = validate(model, val_loader, crit, use_amp)
214     f1m = f1_score(yl, yp, average="macro", zero_division=0)

```

```

215 print(f" Val acc {va:.2f}% | Val F1(macro) {f1m:.4f}")
216 print("\n" + classification_report(y1, yp, target_names=CLASS_NAMES, zero_division=0))
217 plot_confusion(y1, yp, tag, "val"); plot_roc(y1, pr, tag); plot_per_class(y1, yp, tag)
218 tl, ta, *_ = validate(model, train_eval_loader, crit, use_amp)
219 gap = ta - va
220 verdict = "OK (low overfit)" if gap < 5 else ("watch (moderate)" if gap < 10 else "OVERFIT")
221 print(f" Overfit check -> Train {ta:.2f}% / Val {va:.2f}% / Gap {gap:.2f}% -> {verdict}")
222 return va, f1m
223
224
225 def export_util():
226     try:
227         r = subprocess.run([sys.executable, "-m", "pip", "freeze"], capture_output=True, text=True, timeout=60)
228         UTIL_SAVE_PATH.write_text(r.stdout, encoding="utf-8"); print(f" ok util3.txt -> {UTIL_SAVE_PATH}")
229     except Exception as e:
230         print(f" [warn] util3.txt failed: {e} (run: pip freeze > util3.txt)")
231
232
233 def save_checkpoint(model, info, bench):
234     ckpt = {"model_state": model.state_dict(), "class_names": CLASS_NAMES,
235            "use_se": info["use_se"], "use_aux": USE_AUX, "seed": SEED,
236            "val_acc": info["best_val_acc"], "epoch": info["best_epoch"], "benchmark": bench}
237     torch.save(ckpt, MODEL_SAVE_PATH)
238     torch.save(ckpt, MODEL_SAVE_PATH_ROOT)
239     print(f" ok report3.pth -> {MODEL_SAVE_PATH}")
240     print(f" ok report3.pth -> {MODEL_SAVE_PATH_ROOT}")
241
242
243 def main():
244     seed_everything(SEED)
245     tr_csv, va_csv = OUTPUT_DIR / "train_split.csv", OUTPUT_DIR / "val_split.csv"
246     if not tr_csv.exists():
247         print(" [ERROR] run python data_setup3.py first."); sys.exit(1)
248     train_df, val_df = pd.read_csv(tr_csv), pd.read_csv(va_csv)
249     train_loader, val_loader = get_dataloaders(train_df, val_df, BATCH_SIZE)
250     # clean (no-aug) train loader for an honest overfitting check
251     train_eval_loader = DataLoader(VirusDataset(train_df, transform=val_transform),
252                                   batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)
253
254     variants = [True, False] if COMPARE_SE else [True]
255     results, se_model, se_hist, se_info = [], None, None, None
256     for use_se in variants:
257         tag = "SE" if use_se else "noSE"
258         model, hist, info = train_variant(use_se, train_loader, val_loader, train_df, val_df, tag)
259         va, f1m = full_evaluation(model, train_eval_loader, val_loader, hist, tag)
260         bench = benchmark(model)
261         info.update({"val_acc": va, "f1_macro": f1m, **bench})
262         results.append(info)
263         if use_se:
264             se_model, se_hist, se_info, se_bench = model, hist, info, bench
265
266     # save the SE model as the deliverable report3.pth + history
267     se_hist.to_csv(HISTORY_SAVE_PATH, index=False)
268     save_checkpoint(se_model, se_info, se_bench)
269     export_util()
270
271     # comparison table (the exam requirement: with vs without attention)
272     print("\n" + "=" * 92); print(" COMPARISON - WITH vs WITHOUT SE attention"); print("=" * 92)
273     print(f" {'variant':<8}{'val_acc%':>10}{'F1_macro':>10}{'params':>12}"
274           f"{'lat_ms':>10}{'FPS':>9}{'VRAM_MB':>10}{'size_MB':>9}")
275     for r in results:
276         print(f" {r['tag']:<8}{r['val_acc']:>10.2f}{r['f1_macro']:>10.4f}{r['params']:>12,}"
277               f"{'latency_ms':>10.2f}{r['fps']:>9.1f}{r['peak_vram_mb']:>10.1f}{r['model_size_mb']:>9.1f}")
278     with open(OUTPUT_DIR / "se_comparison.json", "w", encoding="utf-8") as f:
279         json.dump(results, f, indent=2)
280     print("\n Saved se_comparison.json. ALL DONE.\n")
281
282
283 if __name__ == "__main__":
284     main()

```